Kazakh-British Technical University

DATABASE

Laboratory Work №2

Student Name: Saken Aibar

ID: 25B030240

18.09.2026

Almaty

Normalization:

Task1

The original `Booking_receipt` table is not in 1NF because the `seat_numbers` column can contain several values in one cell, for example `"12A, 12B"`. In 1NF every column should contain only one atomic value.

After fixing this problem, there are also problems with 2NF and 3NF. For example, `passenger_full_name` depends on `passenger_passport_number`, while flight information depends on `booking_id`. There are also transitive dependencies, because airport cities depend on airport names, and flight information depends on `flight_number`.

So the table should be normalized step by step to remove repeated and dependent data.

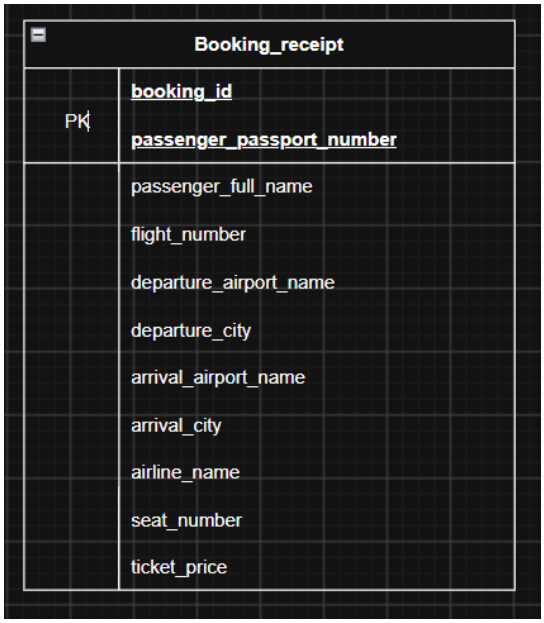
Task2

To bring the table to 1NF, I changed `seat_numbers` to `seat_number`. Each seat is now stored in a separate row, so every column contains only one value.

The primary key is a composite key:

```
(booking_id, passenger_passport_number)
```

This is needed because one booking can contain more than one passenger.

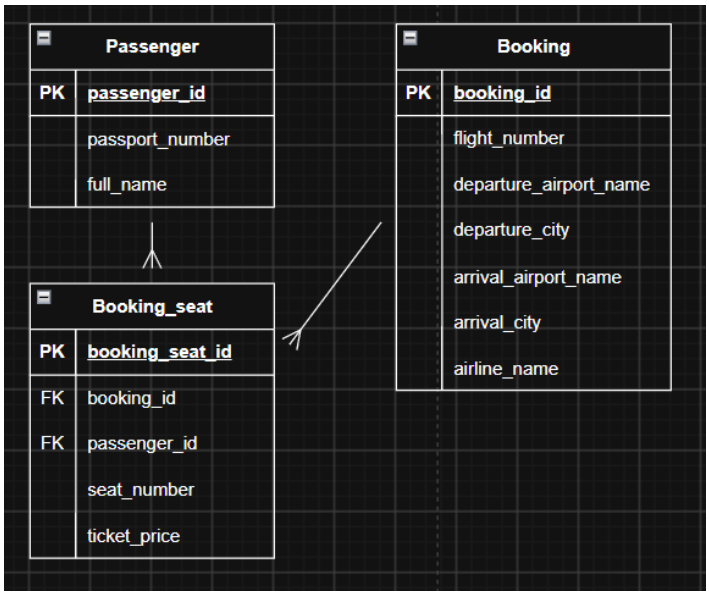


To bring the table to 2NF, I separated data that depends only on part of the composite key.

Passenger information was moved to the `Passenger` table, because passenger data depends on the passenger.

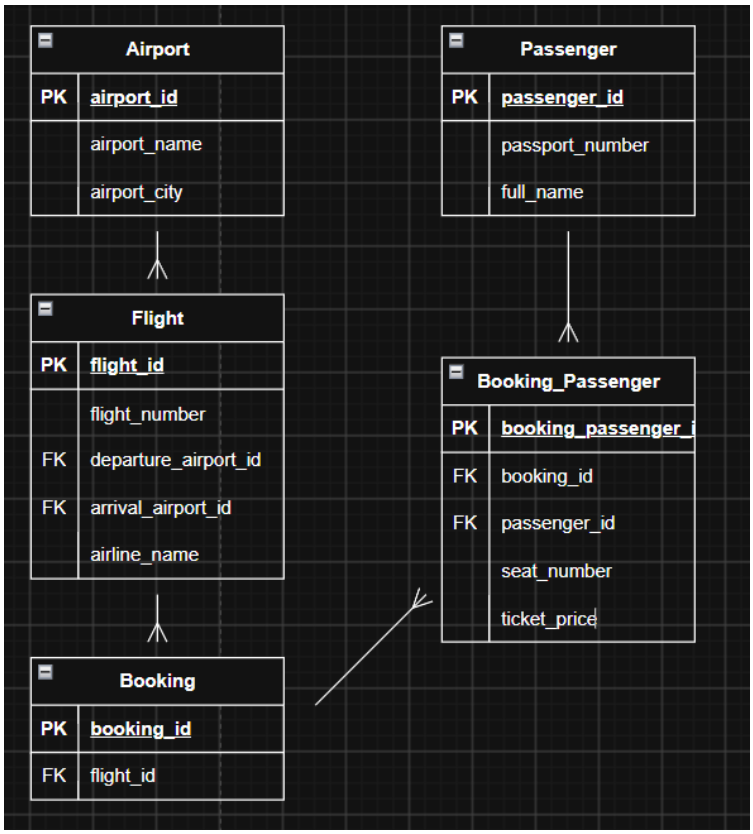
Booking information was moved to the `Booking` table.

The `Booking_Seat` table connects a passenger with a booking and stores information that belongs to this combination, such as `seat_number` and `ticket_price`.



To bring the database to 3NF, I removed transitive dependencies from the `Booking` table.

Flight information was moved to the `Flight` table. Airport information was also moved to the `Airport` table, because the city depends on the airport, not directly on the booking.



Task3

The following functional dependencies were used during normalization:

```
passenger_passport_number → passenger_full_name

booking_id → flight_number

flight_number → departure_airport_name

flight_number → arrival_airport_name

flight_number → airline_name

departure_airport_name → departure_city

arrival_airport_name → arrival_city

(booking_id, passenger_passport_number) → seat_number

(booking_id, passenger_passport_number) → ticket_price
```

These dependencies show which attributes are determined by other attributes.

For example, `passenger_passport_number` determines `passenger_full_name`, so passenger information was moved to a separate `Passenger` table.

Also, `booking_id` determines `flight_number`, while `flight_number` determines flight information such as airports and airline. This creates transitive dependencies, which were removed in 3NF.

Airport cities also depend on airport names:

```
departure_airport_name → departure_city
arrival_airport_name → arrival_city
```

Because of this, airport information was moved to a separate `Airport` table.

The combination of `booking_id` and `passenger_passport_number` determines the passenger's seat and ticket price for that booking.

Task4

The `Flights` table is in 3NF.

The primary key is:

```
flight_id
```

The main functional dependency is:

```
flight_id →
sch_departure_time,
sch_arrival_time,
```

```
departing_airport_id,
arriving_airport_id,
departing_gate,
arriving_gate,
airline_id,
act_departure_time,
act_arrival_time,
created_at,
updated_at
```

All non-key attributes depend directly on `flight_id`.

There is no transitive dependency inside the `Flights` table. For example, `departing_airport_id` and `arriving_airport_id` are foreign keys that refer to the `Airport` table, but airport information such as airport name or city is not stored in `Flights`.

The same applies to `airline_id`. It refers to the `Airline` table, while information such as `airline_name` and `airline_country` is stored there.

Because non-key attributes do not determine other non-key attributes inside the `Flights` table, the table satisfies 3NF.

Task5

The following SQL statements implement the 3NF tables designed in Task 2. The sample data is based on a denormalized booking receipt dataset and is split between the normalized tables.

```
CREATE TABLE Passenger (
    passenger_id INT PRIMARY KEY,
    passport_number VARCHAR(20) UNIQUE NOT NULL,
    full_name VARCHAR(100) NOT NULL
);

CREATE TABLE Airport (
    airport_id INT PRIMARY KEY,
    airport_name VARCHAR(100) UNIQUE NOT NULL,
    airport_city VARCHAR(50) NOT NULL
);

CREATE TABLE Flight (
    flight_id INT PRIMARY KEY,
    flight_number VARCHAR(20) UNIQUE NOT NULL,
    departure_airport_id INT NOT NULL,
    arrival_airport_id INT NOT NULL,
    airline_name VARCHAR(100) NOT NULL,

    FOREIGN KEY (departure_airport_id)
        REFERENCES Airport(airport_id),

    FOREIGN KEY (arrival_airport_id)
        REFERENCES Airport(airport_id)
);

CREATE TABLE Booking (
    booking_id INT PRIMARY KEY,
    flight_id INT NOT NULL,
```

```
    FOREIGN KEY (flight_id)
        REFERENCES Flight(flight_id)
);

CREATE TABLE Booking_Passenger (
    booking_passenger_id INT PRIMARY KEY,
    booking_id INT NOT NULL,
    passenger_id INT NOT NULL,
    seat_number VARCHAR(10) NOT NULL,
    ticket_price DECIMAL(10,2) NOT NULL,

    FOREIGN KEY (booking_id)
        REFERENCES Booking(booking_id),

    FOREIGN KEY (passenger_id)
        REFERENCES Passenger(passenger_id)
);
```

I used a sample denormalized dataset with five booking-passenger records. The data was then split between the normalized tables according to the 3NF structure.

```
INSERT INTO Passenger
(passenger_id, passport_number, full_name)
VALUES
(1, 'N10001', 'John Smith'),
(2, 'N10002', 'Anna Smith'),
(3, 'N10003', 'Michael Brown'),
(4, 'N10004', 'Emma Wilson'),
(5, 'N10005', 'David Lee');


INSERT INTO Airport
(airport_id, airport_name, airport_city)
VALUES
(1, 'Almaty International Airport', 'Almaty'),
(2, 'Astana International Airport', 'Astana'),
(3, 'Shymkent International Airport', 'Shymkent'),
(4, 'Dubai International Airport', 'Dubai');

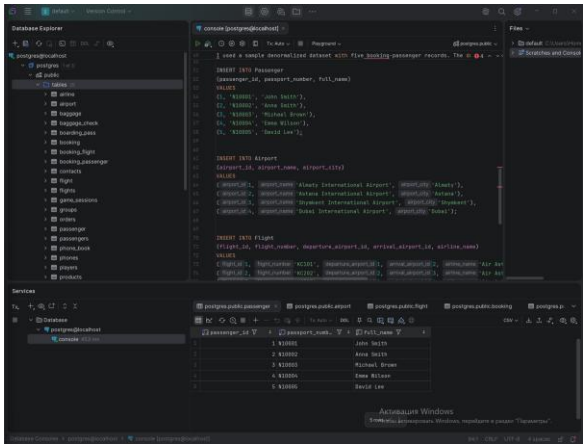

INSERT INTO Flight
(flight_id, flight_number, departure_airport_id, arrival_airport_id,
airline_name)
VALUES
(1, 'KC101', 1, 2, 'Air Astana'),
(2, 'KC202', 2, 3, 'Air Astana'),
(3, 'KC303', 1, 4, 'Air Astana');


INSERT INTO Booking
(booking_id, flight_id)
VALUES
(1001, 1),
(1002, 2),
(1003, 3);


INSERT INTO Booking_Passenger
(booking_passenger_id, booking_id, passenger_id, seat_number, ticket_price)
VALUES
(1, 1001, 1, '12A', 25000.00),
(2, 1001, 2, '12B', 25000.00),
(3, 1002, 3, '7C', 20000.00),
```

```
(4, 1003, 4, '18A', 85000.00),
(5, 1003, 5, '18B', 85000.00);
```

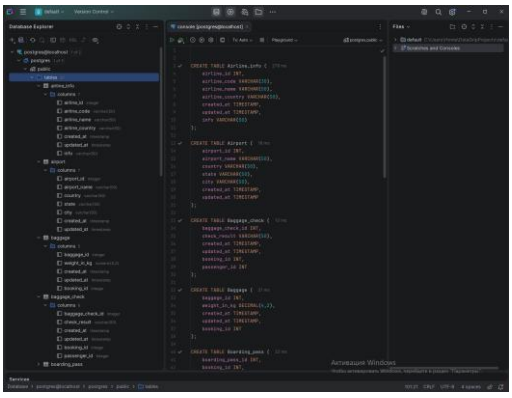
The sample records were successfully inserted into the normalized tables. Each piece of information is stored only in the table where it belongs.

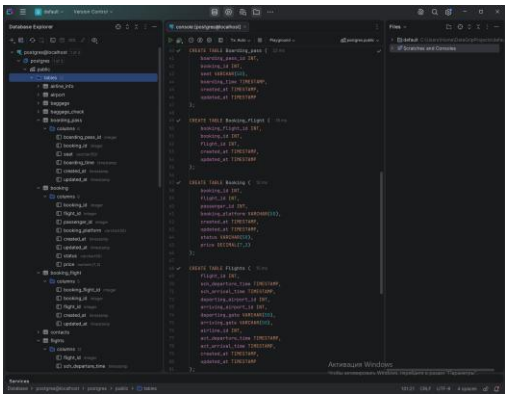


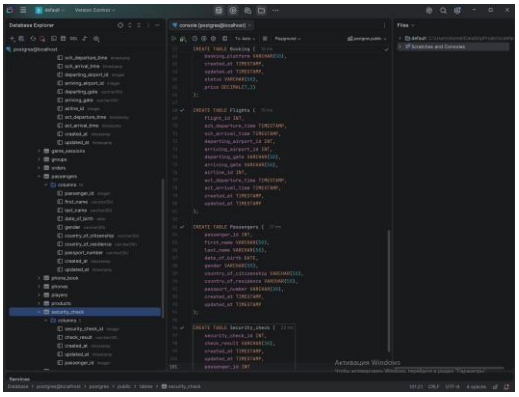
DDL

Task1

The required tables were created with the given attributes and data types. At this step, only the table structure was created without relationships between the tables.

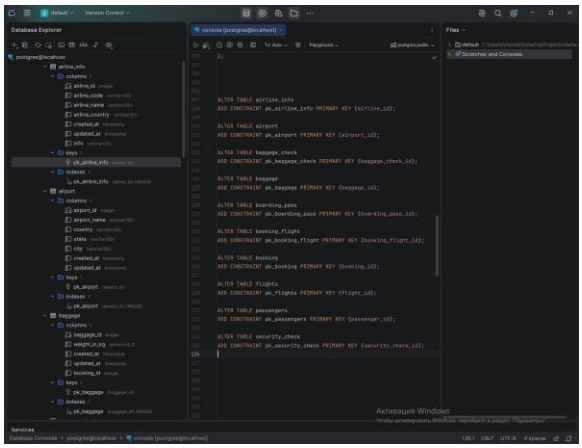






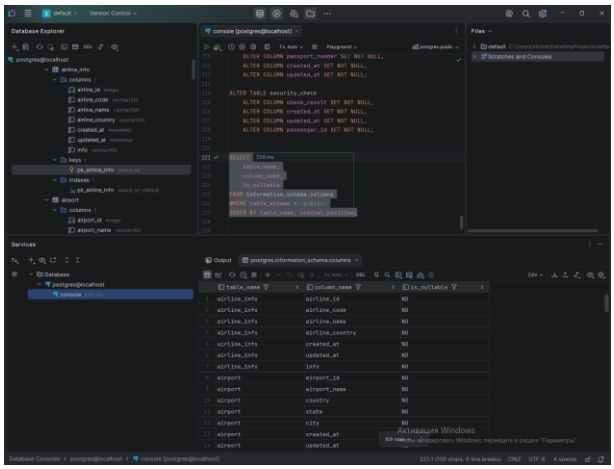
Task2

Primary keys were added to all tables. Each primary key uniquely identifies a row in its table.



Task4

The `NOT NULL` constraint was added to all required attributes. This prevents the columns from containing empty `NULL` values.



Task5

The table `airline_info` was renamed to `airline`.

Task6

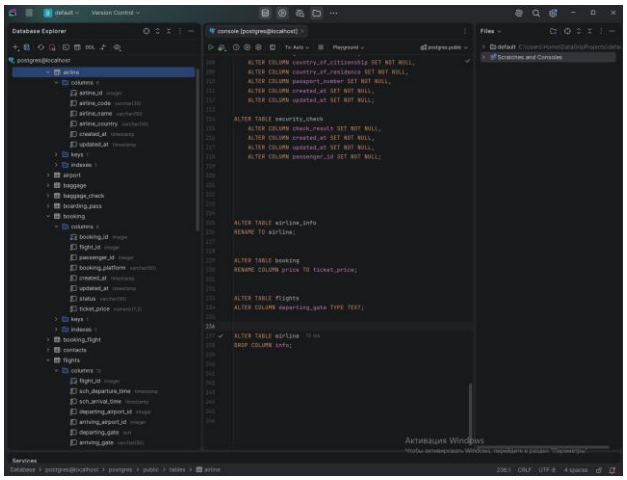
The column `price` in the `booking` table was renamed to `ticket_price`.

Task7

The data type of the `departing_gate` column in the `flights` table was changed from `VARCHAR(50)` to `TEXT`.

Task8

The `info` column was removed from the `airline` table because it is no longer required.



Task9

Foreign keys were added to create relationships between the tables. These relationships connect related records and help maintain data consistency.

The following relationships were created:

```
Passengers 1 : M Security_check
Passengers 1 : M Booking
Passengers 1 : M Baggage_check

Booking 1 : M Baggage_check
Booking 1 : M Baggage
Booking 1 : M Boarding_pass
Booking 1 : M Booking_flight

Flights 1 : M Booking_flight

Airport 1 : M Flights through departing_airport_id
Airport 1 : M Flights through arriving_airport_id

Airline 1 : M Flights
```

The foreign key columns reference the primary keys of the related tables. For example, `passenger_id` in the `security_check` table references `passenger_id` in the `passengers` table.

